

Spring Security

Как обезопасить ваш WEB сервис?



Spring Security

Spring Security это Java/JavaEE framework, предоставляющий механизмы построения систем аутентификации и авторизации, а также другие возможности обеспечения безопасности для корпоративных приложений, созданных с помощью Spring Framework.

Spring Security

Идентификация

Определение, кто автор
запроса

Аутентификация

Удостоверение, что это
тот за кого себя выдает

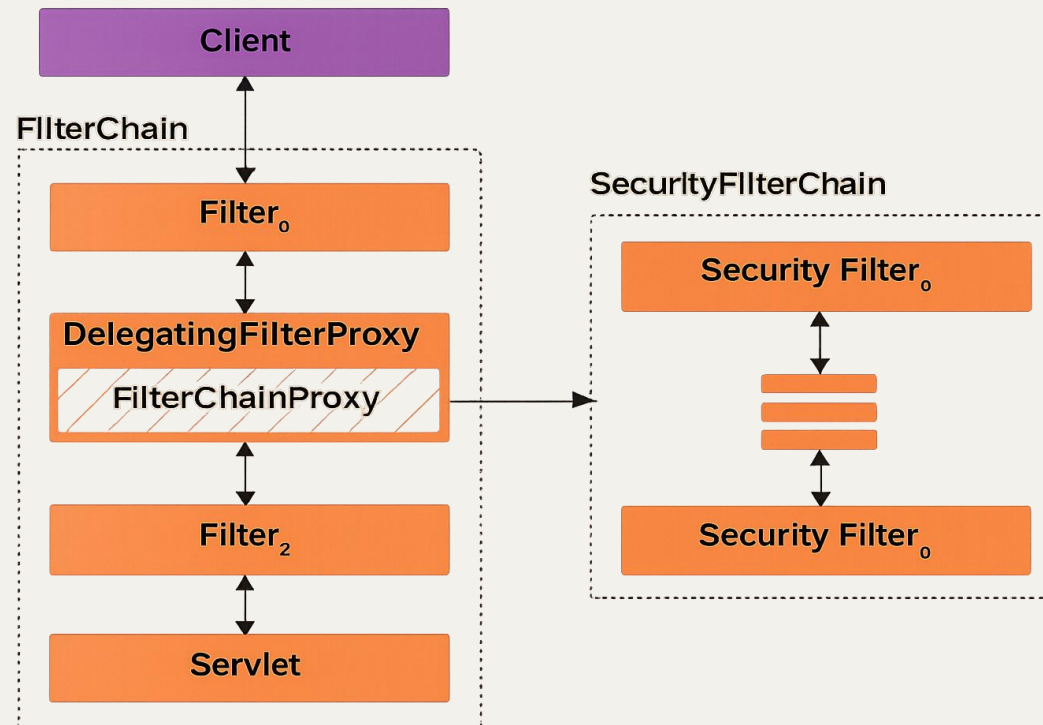
Авторизация

А ему вообще сюда
можно? Определение
прав

Security Filter Chain

Данный проект в своей работе использует паттерн «Цепочки обязанностей». При выполнении http запроса происходит поэтапная обработка фильтрами, которые производят валидацию запроса. Эта цепочка называется **Security Filter Chain**.

Filter Chain



Базовые фильтры безопасности

Фильтры



0 = {WebAsyncManagerIntegrationFilter}
1 = {SecurityContextPersistenceFilter}
2 = {HeaderWriterFilter}
3 = {CsrfFilter}
4 = {LogoutFilter}
5 = {BasicAuthenticationFilter}
6 = {RequestCacheAwareFilter}
7 = {SecurityContextHolderAwareRequestFilter}
8 = {AnonymousAuthenticationFilter}
9 = {SessionManagementFilter}
10 = {ExceptionTranslationFilter}
11 = {FilterSecurityInterceptor}

CATS



- **SecurityContextPersistenceFilter** - Сохранение и восстановление контекста безопасности (SecurityContext) между запросами.
- **CsrfFilter** - Этот фильтр проверяет CSRF-токен в каждом POST-запросе, чтобы убедиться, что запрос был отправлен из доверенного источника.
- **LogoutFilter** - очищает информацию о пользователе из SecurityContext, уничтожает сессию
- **BasicAuthenticationFilter** - Этот фильтр проверяет наличие HTTP-заголовка Authorization с Base64-кодированными учетными данными (логин:пароль)
- **RequestCacheAwareFilter** - Фильтр для восстановления изначального запроса



```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            // отключаем CSRF (например, для REST API)
            .csrf(csrf -> csrf.disable())
            .cors(cors -> cors.disable())

            // настройка авторизации запросов
            .authorizeHttpRequests(auth -> auth.anyRequest().authenticated()) // все требуют
логина
            // включаем форму логина
            .formLogin(form -> form
                .loginPage("/login") // кастомная страница логина
                .permitAll()
            )

            // logout
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
            );

        return http.build();
    }
}
```

Authentication

Authentication — это интерфейс в Spring Security, который представляет собой объект, описывающий текущего пользователя (или клиента) и информацию об его аутентификации. Этот объект хранит данные о пользователе, такие как имя, пароль, роли и другие атрибуты, которые необходимы для проверки прав доступа.

Object getPrincipal()

- Возвращает главный объект (Principal), обычно представляющий пользователя.

Object getCredentials()

- Возвращает учетные данные пользователя, например пароль.

Collection<? extends GrantedAuthority> getAuthorities()

- Возвращает набор прав (GrantedAuthority), которыми обладает пользователь.

boolean isAuthenticated()

- Возвращает true, если пользователь успешно прошел аутентификацию.

AuthenticationManager

AuthenticationManager — это интерфейс в Spring Security, который отвечает за проверку объекта **Authentication**. Он принимает на вход объект **Authentication**, проверяет его валидность и возвращает полностью заполненный объект **Authentication**, если аутентификация успешна. Если проверка не проходит, выбрасывается исключение (**AuthenticationException**).

authenticate(Authentication authentication)

- Принимает объект **Authentication** (например, **UsernamePasswordAuthenticationToken**).
- Возвращает новый объект **Authentication**, если аутентификация успешна.

ProviderManager

ProviderManager — это реализация интерфейса `AuthenticationManager`, которая делегирует процесс аутентификации одному или нескольким провайдерам (`AuthenticationProvider`). Это центральный компонент, который управляет взаимодействием между фильтрами и провайдерами аутентификации.

AuthenticationProvider — это интерфейс, который определяет, как проводится проверка учетных данных для конкретного типа аутентификации. Каждый провайдер отвечает за проверку определенного типа учетных данных.

- **`Authentication authenticate(Authentication authentication)`** : Метод проверяет учетные данные.
- **`bool supports(Class<?> authentication)`** : Метод указывает, поддерживает ли провайдер данный тип объекта `Authentication`.



```
@Component
@RequiredArgsConstructor
public class CustomBasicAuthFilter extends OncePerRequestFilter {

    private final AuthenticationManager authenticationManager;
    private final AuthenticationEntryPoint authenticationEntryPoint;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException
    {
        String header = request.getHeader("Authorization");

        if (header == null || !header.startsWith("Basic ")) {
            filterChain.doFilter(request, response);
            return;
        }

        try {
            UsernamePasswordAuthenticationToken authRequest = extract(header);
            authRequest.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

            Authentication authResult = authenticationManager.authenticate(authRequest);

            SecurityContextHolder.getContext().setAuthentication(authResult);
            filterChain.doFilter(request, response);

        } catch (AuthenticationException ex) {
            SecurityContextHolder.clearContext();
            authenticationEntryPoint.commence(request, response, ex);
        }
    }

    //Извлечение логина и пароля из строки в хедере
    private UsernamePasswordAuthenticationToken extract(String header) ...
}
```

**Давайте попробуем
написать систему вокруг
фильтра**



```
@RequiredArgsConstructor
public class PlainAuthentication implements Authentication {
    private final Long userId;
    private volatile boolean authenticated = true;

    @Override
    public Long getPrincipal() {
        return userId;
    }

    @Override
    public boolean isAuthenticated() {
        return authenticated;
    }

    @Override
    public void setAuthenticated(boolean isAuthenticated) throws IllegalArgumentException {
        authenticated = isAuthenticated;
    }

    ...
}
```



```
@Component
@RequiredArgsConstructor
class DbAuthProvider implements AuthenticationProvider {
    private final UserRepository userRepository;

    @Override
    public Authentication authenticate(Authentication authentication) throws AuthenticationException {
        final var password = authentication.getCredentials().toString();
        if (!"password".equals(password)) {
            throw new AuthenticationServiceException("Invalid username or password");
        }
        return userRepository.findByName(authentication.getName())
            .map(user -> new PlainAuthentication(user.getId()))
            .orElseThrow(() -> new AuthenticationServiceException("Invalid username or password"));
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return UsernamePasswordAuthenticationToken.class.equals(authentication);
    }
}
```



```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            // отключаем CSRF (например, для REST API)
            .csrf(csrf -> csrf.disable())
            .cors(cors -> cors.disable())

            // настройка авторизации запросов
            .authorizeHttpRequests(auth -> auth.anyRequest().authenticated()) // все требуют
логина
            // включаем форму логина
            .formLogin(form -> form
                .loginPage("/login") // кастомная страница логина
                .permitAll()
            )

            // logout
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
            );

        return http.build();
    }
}
```



```
@Entity
@Table(name = "post_role")
public class PostRole {
    @Id
    @GeneratedValue(strategy = IDENTITY)
    private Long id;

    @ManyToOne(fetch = LAZY)
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToOne(fetch = LAZY)
    @JoinColumn(name = "post_id")
    private Post post;

    @Enumerated(STRING)
    private PostRoleType type;
}
```



```
public enum PostRoleType implements Role {  
    VIEWER, EDITOR, REPORTER;  
  
    private final Set<Role> children = new HashSet<>();  
  
    static {  
        REPORTER.children.add(VIEWER);  
        EDITOR.children.add(VIEWER);  
    }  
  
    @Override  
    public boolean includes(Role role) {  
        return this.equals(role) || children.stream().anyMatch(r -> r.includes(role));  
    }  
}
```



```
public enum CommunityRoleType implements Role {  
    ADMIN, MODERATOR;  
  
    @Component("CommunityRole")  
    @Getter  
    static class SpringComponent {  
        private final CommunityRoleType ADMIN = CommunityRoleType.ADMIN;  
        private final CommunityRoleType MODERATOR = CommunityRoleType.MODERATOR;  
    }  
}
```

```

@Service("RoleService")
@RequiredArgsConstructor
public class RoleService {
    private final CommunityRoleRepository communityRoleRepository;
    private final PostRoleRepository postRoleRepository;

    @Transactional
    public boolean hasAnyRoleByCommunityId(Long communityId, Role... roles) {
        final Long userId = ((PlainAuthentication)
SecurityContextHolder.getContext().getAuthentication()).getPrincipal();
        final Set<CommunityRoleType> communityRoleTypes =
            communityRoleRepository.findRoleTypesByUserIdAndCommunityId(userId, communityId);
        for (Role role : roles) {
            if (communityRoleTypes.stream().anyMatch(communityRoleType -> communityRoleType.includes(role))) {
                return true;
            }
        }
        final Set<PostRoleType> postRoleTypes =
            postRoleRepository.findRoleTypesByUserIdAndCommunityId(userId, communityId);
        for (Role role : roles) {
            if (postRoleTypes.stream().anyMatch(postRoleType -> postRoleType.includes(role))) {
                return true;
            }
        }
        return false;
    }

    ...
}

```



```
@RestController
@RequestMapping("/api")
public class Controller {
    @PostMapping("/community")
    @PreAuthorize("isAuthenticated()")
    public CommunityResponse createCommunity(@RequestParam String name) { ... }

    @PostMapping("/community/{communityId}/post")
    @PreAuthorize("@RoleService.hasAnyRoleByCommunityId(#communityId, @CommunityRole.ADMIN)")
    public PostResponse createPost(@PathVariable Long communityId, @RequestParam String name) { ... }

    @PutMapping("/post/{postId}")
    @PreAuthorize("@RoleService.hasAnyRoleByPostId(#postId, @PostRole.EDITOR)")
    public void updatePost(@PathVariable Long postId, @RequestParam String name) { ... }

    @GetMapping("/post/{postId}")
    @PreAuthorize("@RoleService.hasAnyRoleByPostId(#postId, @PostRole.VIEWER)")
    public PostResponse getPost(@PathVariable Long postId) { ... }
```